



DeepCRCEval: Revisiting the Evaluation of Code Review Comment Generation

Junyi Lu^{1,2}, Xiaojia Li³, Zihan Hua^{1,2}, Lei Yu^{1,2}, Shiqi Cheng¹, Li Yang^{1(✉)},
Fengjun Zhang¹, and Chun Zuo⁴

¹ Institute of Software, Chinese Academy of Sciences, Beijing, China
{yulei2022, chengshiqi, yangli2017, fengjun}@iscas.ac.cn

² University of Chinese Academy of Sciences, Beijing, China
{lujunyi21, huazihan22}@mails.ucas.ac.cn

³ Kuaishou Technology, Beijing, China

lixiaojia03@kuaishou.com

⁴ Sinosoft Company Limited, Beijing, China
zuochun@sinosoft.com.cn

Abstract. Code review is a vital but demanding aspect of software development, generating significant interest in automating review comments. Traditional evaluation methods for these comments, primarily based on text similarity, face two major challenges: inconsistent reliability of human-authored comments in open-source projects and the weak correlation of text similarity with objectives like enhancing code quality and detecting defects.

This study empirically analyzes benchmark comments using a novel set of criteria informed by prior research and developer interviews. We then similarly revisit the evaluation of existing methodologies. Our evaluation framework, DeepCRCEval, integrates human evaluators and Large Language Models (LLMs) for a comprehensive reassessment of current techniques based on the criteria set. Besides, we also introduce an innovative and efficient baseline, LLM-Reviewer, leveraging the few-shot learning capabilities of LLMs for a target-oriented comparison.

Our research highlights the limitations of text similarity metrics, finding that less than 10% of benchmark comments are high quality for automation. In contrast, DeepCRCEval effectively distinguishes between high and low-quality comments, proving to be a more reliable evaluation mechanism. Incorporating LLM evaluators into DeepCRCEval significantly boosts efficiency, reducing time and cost by 88.78% and 90.32%, respectively. Furthermore, LLM-Reviewer demonstrates significant potential of focusing task real targets in comment generation.

Keywords: Code review automation · Evaluation framework · Large language models (LLMs) · Text similarity metrics · Defect detection

An extended version of this paper, which includes appendices is available at
<https://doi.org/10.48550/arXiv.2412.18291>

1 Introduction

Since Fagan’s pioneering work on software inspections in 1976 [7], code review practices have evolved significantly. Early software inspections were formal and resource-intensive, which limited their widespread adoption in the industry. In contrast, Modern Code Review (MCR) has become a vital and streamlined process in both Open-Source Software (OSS) [25–27] and industrial applications [28, 29]. MCR typically involves developers submitting code changes for review, which reviewers then assess to provide feedback on potential issues or improvements. This process may involve mechanisms such as pull requests, change-sets, or code patches, depending on the tools and platforms used. A pivotal aspect of MCR is the generation of insightful, constructive comments, essential for guiding developers and pinpointing problems, which is further elaborated upon in the section ‘Additional Background’ in the appendix of our extended version.

Despite its effectiveness, MCR remains resource-intensive [34]. This challenge has spurred the exploration of automating code review comments to alleviate labor demands [11]. Initial attempts focused on retrieval-based methods using existing comments as references [8, 30]. With the progression of deep learning, the emphasis has shifted to generative approaches. A variety of Code Review Comment Generators (CRCGs) have emerged, notably initiated by Tufano et al. with a T5 transformer architecture [32], later augmented with code-technical language pre-training [31]. Following this trend, models like CodeReviewer [16] and AUGER [14] have further advanced the field, integrating specific pre-training for code review and utilizing review tags to enhance accuracy. Parallel developments include CommentFinder [10], offering an efficient retrieval-based solution, and CCT5 [17], which underscores the importance of considering code changes in comment generation. Later, with the emergence of LLMs, Lu et al. [20] investigate the factors influencing the code review process for both traditional pre-trained language models and LLMs, while Llama-Reviewer [21] marks the first attempt at training code review tasks on LLMs in a parameter-efficient approach. Nevertheless, these models typically employ text similarity metrics such as BLEU [22] and ROUGE [18] for evaluation.

We question the suitability of text similarity as the primary metric for assessing code review comment automation. The dependability of human-written comments in OSS, commonly used as benchmarks, is often subject to scrutiny [1–3, 9, 12, 23, 33]. These comments can be arbitrary and inconsistent, at times offering little more than basic queries or directives like “Why do we need this?” or “Remove this.” Unlike traditional text-to-text tasks such as summarization or translation, which seek semantic equivalence, code review comments aim to aid in defect detection and code refinement, requiring deeper insight [1]. Hence, text similarity, as an indirect measurement, falls short of capturing the essence.

Analysis of Benchmark Comments. The reliability of text similarity metrics for evaluating review comments hinges on the accuracy of the reference text. Yet, the quality and validity of benchmark comments in major datasets, such as the CodeReviewer (CRer) [16] and Tufano datasets [31], often remain ambiguous. To address these ambiguities and provide a more nuanced understanding, we

conducted an empirical study analyzing these datasets from four dimensions: 1) Quality: Assessing comments against established quality standards; 2) Category: Evaluating the effectiveness of comments in identifying defects and suggesting improvements, or other potential roles; 3) Tone: Analyzing whether comments clearly state issues or are merely interrogative; 4) Context: Determining if the comments are sufficiently supported by the associated code. For Quality evaluation, we developed criteria based on existing literature about developers’ views on code review quality [12], supplemented by our semi-structured interviews and card sorting exercises using affinity diagrams.

DeepCRCEval Stemming from our analysis of benchmark comments, we developed DeepCRCEval, an innovative evaluation framework incorporating both human and LLM evaluators. This framework utilized the criteria of high quality comments identified in the former analysis, and use both scoring and ranking to compare performances. It is geared towards identifying the intrinsic merits of review comments, moving away from the indirect approach of text similarity.

LLM-Reviewer To validate the effectiveness of direct metrics in line with code review objectives, we propose LLM-Reviewer, a lightweight, training-free baseline tool. Traditional methods, reliant on text similarity metrics, may not capture the true essence of code review. In contrast, by harnessing the potential of Large Language Models (LLMs), LLM-Reviewer employs few-shot learning with meticulously crafted prompts. Unlike previous methods, LLM-Reviewer directly addresses the actual aims of code review and controlled the comments’ quality with criterion guidance in prompt, more closely mirroring developers’ concept of effective review commentary. This baseline also serves as a benchmark to gauge the effectiveness of evaluation frameworks in differentiating high and low-quality review comments.

Revisiting the Evaluation of CRCGs Utilizing DeepCRCEval as our evaluation framework and LLM-Reviewer as our baseline, we reassessed the performance of current state-of-the-art (SOTA) CRCGs. Given the training-free nature of LLM-Reviewer, it was anticipated that other tuned methods would perform at least comparably. However, our findings, derived from both human and LLM evaluators, reveal a considerable shortfall in the performance of existing SOTA CRCGs compared to this benchmark. Our analysis of 1,000 typical code snippets containing defects or code smells demonstrated LLM-Reviewer’s proficiency in consistently pinpointing issues, providing well-rounded comments encompassing problem identification, detailed explanation, and possible solutions. In contrast, SOTA CRCGs often produced nonspecific and ambiguous comments, falling short of the review’s goals.

Contributions Our significant contributions include:

1. Unveiling the biases in evaluating state-of-the-art (SOTA) CRCGs due to reliance on inappropriate metrics.
2. Developing DeepCRCEval, a versatile evaluation framework employing either human or LLM evaluators, which concentrates on the core essence of comments rather than indirect text similarity measures.

3. Introducing LLM-Reviewer, a pioneering, training-free baseline for code review comment automation.
4. Empirically demonstrating that existing SOTA CRCGs are outperformed by the training-free LLM-Reviewer, indicating potential for improvement.
5. Materials publicly available at <https://doi.org/10.5281/zenodo.10511726>.

The rest of the paper is structured as follows: Section 2 presents the background of our research. Section 3 introduces the overview of the study and our research questions. Section 4 details our analysis of benchmark comments, which aims to prove the limitations of current text similarity measurements. Section 5 illustrates our approach to revisiting the evaluation of code review comment generators, including our dual-granularity human/LLM evaluation framework DeepCRCEval, and a prompt-based LLM baseline LLM-Reviewer. Section 6 shows the results of the reevaluation, as well as key findings and discussions. Section 8 introduces related work on the evaluation of code review comments and the differences between these and our work. Section 9 concludes the paper.

2 Background

2.1 CRCGs and Their Evaluation

For code review automation, researchers predominantly utilize deep learning or information retrieval techniques as code review comment generators (CRCGs) to automatically generate comments for given code snippets. Despite some variations, Figure 1 depicts the typical workflow for training a deep neural network (DNN) model or constructing a retriever for automating code review. Initially, ① a DNN model or a retriever is trained or established using a dataset of code-comment pairs from OSS projects, learning the semantic relationships between code and comments. Then, ② for each test case code snippet, the model or retriever generates or retrieves a comment employing specific decoding or retrieval techniques. Finally, ③ the produced comments are compared against the original ones from the test set, also sourced from OSS projects. Common comparison metrics include BLEU [22] and ROUGE [18], where BLEU assesses the precision of machine translations by measuring the match of N-grams, and ROUGE evaluates the recall of machine-generated summaries based on N-gram co-occurrence. These metrics also guide the training of DNNs.

However, the quality and validity of the original comments extracted from OSS projects are questionable. For example, the comments in Table 1 are not suitable for use as code review comments by **models**, although they might be meaningful for human reviewers. Specifically, “Why do we need this?” is indeed meaningful if it opens a dialogue when by humans. However, CRCGs generate comments only once, without a dialogue-like interaction. If a model outputs this comment, the code author cannot acquire further knowledge as the process has concluded. “Remove this line” provides specific action suggestions. However, without reasons, it could confuse code authors. If by humans, this could be

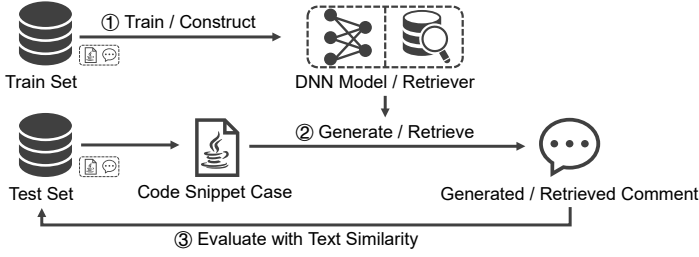


Fig. 1. The overall workflow of learning a deep neural network (DNN) model or retriever to automate code review.

clarified by asking further questions, but for CRCGs the process is already complete. Besides, these two comments are both too general. They lack context and explanation, making it applicable to any code.

Moreover, the text similarity used for comparison are indirect measures of the primary objective, which is to find defects or enhance code. Unlike tasks such as text translation where similar input leads to similar output, review comments can be arbitrary. A same issue can be represented differently, while similar representations might have totally different meanings. Therefore, we posit that there might be an overestimation of the effectiveness of current SOTA CRCGs.

Table 1. Example of unsuitable comments for machine code review.

It's a race to see who merges first, because I bet one of my PRs will cause a conflict with this
why do we need this
remove this line
While I would have liked to resolve all of the N+1 issues related to profiles in this PR this has proved more difficult than anticipated and there's a lot of other good stuff in this PR that I'd really like to get out, so we can keep this around a bit longer.
Can we avoid change names of variable for now? This change is not purely style change and might make those who maintains private patches harder. Such change can be done separately with a more fine grained approach.
I seem to remember we spoke on this earlier in the opposite sense, saying that we want the shortcuts to be added even if they are not displayed.
The same code has been used a few times throughout this file.

2.2 Large Language Models for Evaluations

Large language models have shown capabilities similar to those of human evaluators [13]. Studies have demonstrated that large language models like GPT-4 can achieve higher agreement than human evaluators [6, 15, 24, 35] for ranking preference and scoring. For instance, in the evaluation of general text generation, GPT-4's agreement rate with human evaluations (85%) surpasses the rate of inter-human agreement (81%) [35], a finding echoed in another dataset comparison (69.2% vs. 65.7%) [15]. Considering this, our evaluation framework is based

on these works but is more granular and lightweight. We use a chain-of-thought template to inject task-specific knowledge for better scoring and ranking.

Since there is no prior proof of the effectiveness of LLM evaluators in the task of code review comment generation, we adopt both human and LLM evaluators for all evaluation parts of our paper. The LLM evaluators serve as an auxiliary certification, expanding the scope of verification.

3 Overview and Research Questions

Automated code review has garnered significant interest from researchers, yet its evaluation effectiveness has not been sufficiently addressed. This oversight leads to a disconnect between the task objectives and the training processes used. This paper aims to highlight the importance of effective evaluation in code review automation. As depicted in Figure 2, we developed ① DeepCRCEval, a new evaluation framework incorporating both human and LLM evaluators, and ② LLM-Reviewer, a lightweight, training-free, and target-oriented baseline tool. Our study focuses on: **1)** verifying the shortcomings of current evaluation metrics, **2)** manually reassessing the evaluation of existing code review comment generators, **3)** integrating LLM-based alternative evaluators for an extended scope of validation, and **4)** identifying potential improvement directions stemming from the misalignment of task objectives and training processes. The latter two RQs aim to set directions for future research. We next introduce the research questions we aim to investigate and their relationships.

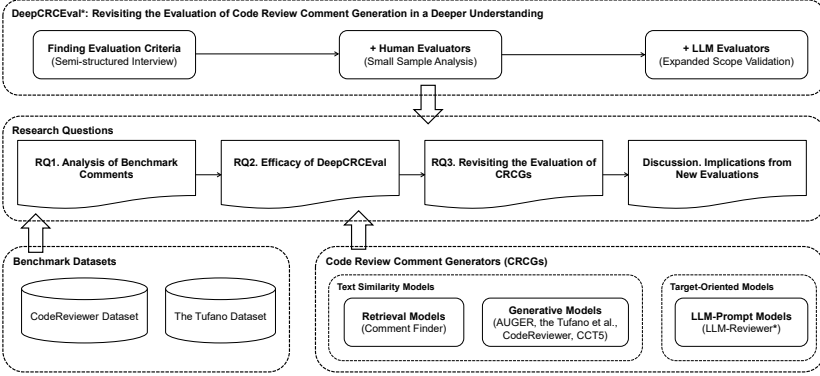


Fig. 2. Overview of our study. * indicates frameworks or models we newly proposed.

RQ1. Analysis of Benchmark Comments: Are the foundations of current evaluation metrics reliable? The reliability of text similarity metrics for evaluating review comments depends on the quality and validation of the reference texts. We first analyze the benchmark reference review comments from

four perspectives: quality, category, tone, and context. We present results for each aspect and summarize the relationships between these aspects.

RQ2. Efficacy of DeepCRCEval: Why do we claim that DeepCRCEval provides a deeper evaluation, and why do we integrate LLM evaluators? Before presenting the new results obtained in RQ3, we first examine the differences between our DeepCRCEval and traditional text similarity metrics. Additionally, we evaluate the advantages and disadvantages of using LLM evaluators for expanded scope validation. We report on the efficiency of LLM evaluators and their agreement with human evaluators.

RQ3. Revisiting the Evaluation of CRCGs: What are the actual performances of current CRCGs beyond simple text similarity metrics? By challenging the current text similarity metrics in RQ1, we aim to investigate the actual performances of current CRCGs more deeply. Using our newly proposed evaluation framework, DeepCRCEval, and the new target-oriented model, LLM-Reviewer, as a baseline, we increase the distinction in our analyses. We analyze results from human evaluators as a small sample analysis and from LLM evaluators for expanded scope validation.

Discussion. Implications from New Evaluations: What can we learn to guide further research? The new evaluations aim to guide future research in code review comment generation and propose potential improvement directions. We present the implications derived from our study with case studies.

4 RQ1. Analysis of Benchmark Comments

4.1 Aspects for Analysis

To thoroughly explore previous code review comment datasets, we define aspects with a qualitative and quantitative process [4], which draws on varied data sources for comprehensive insights, including the review of previous study [12], our semi-structured interviews [19] with seven industry developers, and subsequent card sort and affinity diagram by the authors of this paper. The detailed process are shown in the section ‘Finding Quality Evaluation Criteria’ in the appendix of our extended version. These gained criteria are utilized for assessing benchmark comment quality, which will also be for the subsequent reevaluation.

Quality The multi-step study introduced above culminated in identifying nine key criteria for evaluating the effectiveness of code review comments:

- C1. Readability:** Clear, easily understandable language.
- C2. Relevance:** Directly related to the specific code.
- C3. Explanation Clarity:** Clear elucidation of the issues identified.
- C4. Problem Identification:** Accurate pinpointing and articulation of bugs.
- C5. Actionability:** Practical advice for addressing identified issues.
- C6. Completeness:** Coverage of all issues in the code for comprehensive review.
- C7. Specificity:** Focus on specific code issues, avoiding generic statements.
- C8. Contextual Adequacy:** Comments pointing out exact issue locations.
- C9. Brevity:** Conciseness, conveying essential information without verbosity.

Category We evaluated the comments’ ability to detect defects or suggest code improvements using a classification system. This involved adopting the nine categories proposed by Bacchelli et al. [1], such as Code Improvement, Understanding, and Social Communication, and introducing an additional category, “Meaningless Text”, for extremely low-quality, uninformative comments.

Tone and Context To examine factors influencing comment validation, we conducted a manual inspection using the Nominal Group Technique (NGT), involving three authors of this paper. Each participant initially prepared individual opinions, later discussed in a structured meeting. The focus was on the aspects with the highest consensus:

- ① **Tone:** Interrogative comments were noted to be less effective, as they often raise questions rather than providing specific, formalized feedback. For example, a comment like “Why this?” is less valuable for identifying defects or suggesting improvements.
- ② **Context:** Comments requiring understanding beyond the provided code snippet (e.g., at the file level) were found challenging, especially in the context of automated code review techniques and their datasets. Comments such as “used a few times throughout this file” lack clarity when only a single method is given as input.

4.2 Analysis Methodology

Analyzing code review comments is a nuanced and labor-intensive task. To manage this, we sampled 100 comments from each of the two primary datasets in this domain: the Tufano dataset [31] and the CodeReviewer dataset [16], abbreviated as Tufano and CRer, respectively. According to the average reliability of 93% for humans in Table 6, the margin of error for 95% confidence level of 100 sample size is within 5%. The Tufano dataset is a monolingual, function-level Java dataset, while the CRer dataset is multilingual and at the diff granularity. Both are constructed from large-scale open-source software repositories and widely utilized in numerous studies. The quality and category is finished with a human scoring system created using QT and a Delphi Method variant [5] by five master’s and doctoral students, respectively. The Tone and context was conducted using the aforementioned NGT sessions by three authors of this study. The detailed process are shown in the section ‘Detailed Analysis Methodology for Datasets Comment Quality’ in the appendix of our extended version.

4.3 Results of Analysis

We first present the results from each aspect, and then summarize the relationships between each aspect using a Venn diagram in Section 4.3.

Quality Our analysis, illustrated in the upper part of Table 2, indicates that while OSS review comments typically exhibit good readability and brevity, they frequently lack in other critical aspects, reflecting issues of arbitrariness, incompleteness, and irregularity. We designated scores **below 6 (out of 10)** as poor performance indicators for each aspect. The lower part of Table 2 shows the proportion of comments scoring poorly in each aspect. This data highlights deficiencies of comments in aspects other than readability, completeness, and brevity. For example, low scores in explanation clarity or actionability hint at inadequate detail or absence of constructive suggestions, while low relevance, contextual adequacy, or specificity scores suggest a tendency towards vagueness or irrelevance to the code context. LLM evaluations were also conducted, similar to Section 5.1, with detailed results available in Table ‘Average quality of comments and percentage of low-quality cases in OSS datasets by LLM evaluators’ in the appendix of our extended version.

Table 2. Average quality of comments (① upper part, 1-10) and percentage of low-quality cases (② lower part, 0%-100%) in OSS datasets. C1-C9 represent aspects mentioned in Section 4.1.

	Dataset	C1	C2	C3	C4	C5	C6	C7	C8	C9
①	Tufano	8.68	7.07	4.70	5.40	5.81	7.61	6.43	4.89	8.99
	CRer	9.21	6.41	5.23	5.78	5.08	7.40	6.36	6.32	9.03
②	Tufano	7%	31%	64%	55%	46%	20%	34%	63%	4%
	CRer	1%	48%	57%	52%	64%	31%	35%	43%	0%

Category The classification of comment categories in OSS projects, summarized in Table 3, reveals distinct distributions. In the Tufano dataset, code improvements and defects constitute 64%, while in the CRer dataset, they comprise only 39%. This distribution aligns with previous findings on the proportion of practically useful comments [3]. Comments outside these categories may hold value in specific human code review scenarios but do not align with the core objectives of machine code reviews. For example, comments on deferring tasks to future pull requests, unrelated to the current code, offer limited value.

Tone and Context The analysis shows a significant presence of interrogative comments—38% in the Tufano dataset and 46% in the CRer dataset. Besides, a substantial portion of comments—45% in the Tufano dataset and 54% in the CRer dataset—required out-of-method or out-of-hunk context.

Summary of Analysis The Venn diagrams in Figure 3 for each dataset summarize the interplay of various factors impacting the suitability of reference comments. A notable finding is that only a small fraction of comments in these datasets—3% in the Tufano dataset and 8% in the CRer dataset—qualify as ideal

Table 3. Distribution of comment categories in OSS datasets.

	the Tufano dataset	the CRer dataset
Code Improvement	43%	26%
Understanding	5%	19%
Social Communication	12%	18%
Defects	21%	13%
External Impact	2%	1%
Testing	2%	3%
Review Tool	2%	1%
Knowledge Transfer	1%	8%
Misc	1%	3%
Meaningless Text	11%	8%

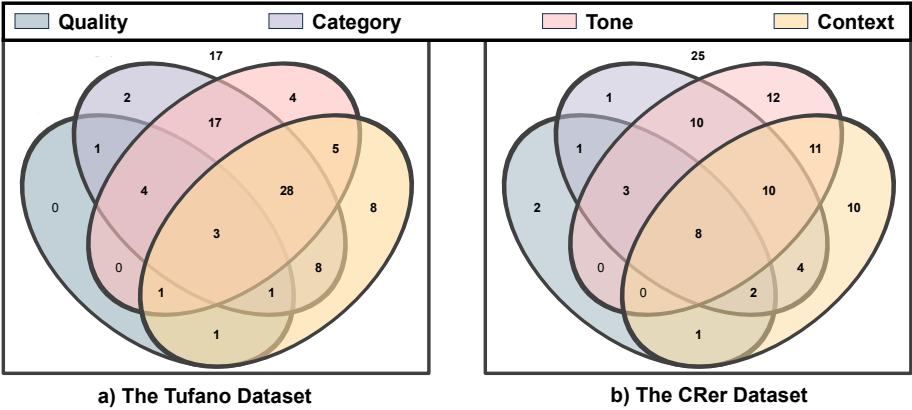


Fig. 3. 4-group Venn diagrams showing the overlap of suitable quality, category, tone, and context in comments.

references. Furthermore, even within this subset, many comments only address minor issues like typos or simple syntax errors.

Response to RQ1: The uncertain quality and validity of benchmark comments undermine their suitability as reference standards, casting doubt on the effectiveness of text similarity as a metric.

5 Revisiting the Evaluation of CRCGs

In light of the inadequacy of text similarity metrics for evaluating Code Review Comment Generators (CRCGs), this section introduces our novel evaluation framework, DeepCRCEval, and a target-oriented baseline, LLM-Reviewer, to reassess the performance of CRCGs.

5.1 DeepCRCEval

Drawing from the criteria established in Section 4.1, we developed DeepCRCEval, an evaluation framework designed to rigorously analyze the quality of generated comments. DeepCRCEval employs both human and Large Language Model (LLM) evaluators. Initially, we conducted human evaluations on a sampled test set due to cost considerations, then extended the evaluation to the entire test set using LLMs.

Human Evaluators For human evaluations, we developed an executable file using QT, similar to the one in Section 4.3 but enhanced with a comparative ranking task. This tool is also available in our open-source repository.

LLM Evaluators Recognizing the limitations of human scoring, such as cost and time intensity, and challenges in assessing certain aspects like completeness, we incorporated LLM evaluators. Our methodology utilizes a thought-chain-enhanced prompt template, adapted from recent research [35]:

- **Domain-Based Scoring:** In contrast to earlier methods that used a single overall score, we introduced domain-specific scoring across nine different dimensions, allowing for a more detailed and nuanced assessment.
- **Efficient Sorting:** We optimized resource usage by streamlining pairwise comparisons into a consolidated overall sorting, simplifying the process.
- **Chain-of-Thought Integration:** Chain-of-thought (CoT) module was embedded to link scoring and ranking tasks coherently, enhancing the logical flow of evaluations.

The structure of our enhanced prompt template is:

$$P_{Eval} = Des_{Scoring} + G + Obj + Des_{Ranking} + F \quad (1)$$

$$F = F_{Scoring} + CoT + F_{Ranking} \quad (2)$$

where $Des_{Scoring}$ and $Des_{Ranking}$ denote the task descriptions for scoring and ranking, respectively. G comprises guidelines, including notes and criteria descriptions, while Obj represents the objects being evaluated. F outlines the format for the generation, integrating a CoT section for enriched explanations. We delineate each component with “###” for clarity. The detailed prompt template is shown in Table ‘Detailed prompt template for LLM evaluators’ in the appendix of our extended version. To minimize bias, evaluations were performed twice for each case, once in descending and once in ascending order.

5.2 Selected CRCGs

For our analysis, we first selected all current state-of-the-art (SOTA) CRCGs published in top-tier conferences. This includes models like Tufano et al. [31],

AUGER [14], CodeReviewer [16], and CCT5 [17], primarily based on DNN models, alongside CommentFinder [10], which uses retrieval techniques. Notably, all these CRCGs have been trained or constructed using data from OSS projects and guided by text similarity metrics, which may not align well with the primary goal of identifying defects or enhancing code.

To evaluate CRCG performance effectively, an appropriate baseline is crucial. We sought a baseline that directly targets defect detection and code improvement. Given the absence of such a baseline in existing literature, we introduce LLM-Reviewer, a novel, straightforward, and fair baseline. It resembles DNNs but does not require additional training.

5.3 LLM-Reviewer

Figure 4 illustrates the workflow of LLM-Reviewer. Distinct from traditional CRCGs, LLM-Reviewer operates without the need for a training set. The process for each new code snippet is: ① Integration of the code with a pre-defined prompt. ② Feeding this combined input into the selected LLM to generate a comment. ③ Direct evaluation of the generated comment’s quality using DeepCRCEval.

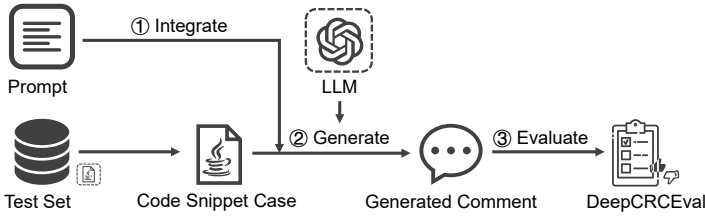


Fig. 4. The overall workflow of LLM-Reviewer.

The pre-defined prompt is bolstered with specific guidance and few-shot instructions. The utilized in-context prompt is:

$$P_{Reviewer} = Des_{Gen} + G_{Gen} + Demo^k \quad (3)$$

where Des_{Gen} offers concise directives for generating code review comments, G_{Gen} encompasses guidelines including notes and criteria descriptions (as discussed in Section 4.1), and $Demo^k$ features k randomly chosen demonstrations, adhering to k -shot learning principles. For this study, we opted for $k = 3$, balancing input length with informative content. Each prompt component is clearly separated by “###” as detailed in Table ‘Detailed prompt template for LLM-Reviewer’ in the appendix of our extended version.

This prompt is then integrated with the code snippet under review before being fed into the LLM for comment generation. It is worth noting that we use a similar prompt template to the one used for prompt evaluation. This similarity is why we refer to LLM-Reviewer as a target-oriented model. Unlike other

text similarity models, the similar prompt template constrains LLM-Reviewer’s response to the expected target. Additionally, as a countermeasure against bias, we use human evaluators throughout the entire study.

5.4 Other Experimental Details

Testing Set Since LLM-Reviewer does not require a training dataset, we selected a set of 1,000 code cases with typical issues for testing. These cases are processed by humans to enhance simplicity, and thus to reduce the risk of data leakage. Each case is guaranteed to contain at least one significant issue within one or a few functions, compatible with the input format of earlier CRCGs. We used ROUGE-L to remove duplicates. This test set is also accessible in our open-source repository. For baselines, we utilized their respective training sets.

Implementation of Baselines The implementation of baseline CRCGs involved different approaches. For Tufano et al. [31], AUGER [14], and CCT5 [17], we leveraged their publicly available models. CommentFinder [10] was replicated using the original dataset and code, while CodeReviewer [16] was obtained through fine-tuning their pre-trained model with provided scripts. All GPT-4 usages within the DeepCRCEval framework and our LLM-Reviewer were implemented using the OpenAI Python library, with a temperature setting of 0.1 and a token limit of 8192.

6 RQ2-RQ3: Empirical Findings

This section presents our empirical findings, detailing the performance of various CRCGs as evaluated by our framework DeepCRCEval.

6.1 RQ2. Efficacy of DeepCRCEval

This subsection examines the efficacy of our DeepCRCEval framework, specifically assessing the criteria used for evaluation and the integration of LLMs.

Effectiveness of Criteria Our evaluation framework, DeepCRCEval, surpasses traditional text similarity metrics in two key areas: discrimination and comprehensiveness.

Discrimination Prior studies often reported marginal improvements in text similarity metrics, like a less than 1% increase in BLEU scores. Such negligible enhancements do not reliably indicate an improvement in comment quality, as corroborated by our reevaluations. While newer baselines like CCT5 and AUGER reported improvements in text similarity, they did not surpass their predecessor, Tufano et al., in effectiveness—a conclusion also supported by our qualitative case studies. In contrast, DeepCRCEval, with its well-defined criteria, offers a higher degree of discrimination across various aspects. Moreover, our ranking process provides a direct and comparative analysis of comment quality.

Table 4. Comparison of average time and cost per case per evaluator between human and LLM evaluators.

Evaluator	Single Comment		Performance Comparison	
	Time (s)	Cost (\$)	Time (s)	Cost (\$)
Humans	224.45	0.62	752.65	2.09
LLMs	25.18	0.06	68.69	0.17

Table 5. Levels of agreement between humans and LLMs.

Evaluators	C1	C2	C3	C4	C5	C6	C7	C8	C9
Humans	0.89	0.94	0.94	0.94	0.95	0.91	0.94	0.95	0.90
LLMs	0.62	0.68	0.83	0.80	0.81	0.72	0.79	0.78	0.62

Comprehensiveness DeepCRCEval’s second advantage is its ability to offer a holistic evaluation. Unlike previous studies that could not elucidate why their methods were superior using text similarity metrics, DeepCRCEval, by incorporating domain-specific criteria, sheds light on the strengths and weaknesses of different models, providing a more rounded assessment.

Efficacy of LLM Evaluators Integrating LLMs as evaluators aims to enhance automation and minimize cost.

Efficiency To demonstrate the efficiency of LLM evaluators compared to human evaluators, we analyzed the average time and cost per case. Human evaluators were paid \$10 per hour, while LLM evaluations were costed based on API charges (\$0.03 per 1000 input tokens, \$0.06 per 1000 output tokens). Table 4 contrasts the average time and cost for both single comment evaluations and comparative performance analyses. The results underscore the significant reduction in time and cost achieved with LLM evaluators.

Effectiveness We utilized the Inter-class Correlation Coefficient (ICC) to gauge the concordance between LLM and human evaluators, setting the average human scores as the reference. Table 5 details their agreements across various evaluation aspects. Specifically, for each quality metric, two coefficients are reported: one represents the agreement between human evaluators and the reference scores (Human vs. Reference), and the other captures the agreement between LLM outputs and the reference scores (LLM vs. Reference). Despite a marginally lower agreement in some areas, the LLM evaluators achieved high concordance (above 0.75) in aspects like Explanation Clarity, Problem Identification, Actionability, Specificity, and Contextual Adequacy. In aspects where LLM and human evaluators diverged, we observed distinct scoring tendencies. For example, humans were prone to extreme ratings in Readability and Relevance, while LLMs offered more evenly distributed scores. LLMs also considered grammatical and syntactic factors in Brevity, unlike humans who focused on content length and relevance.

Response to RQ2: DeepCRCEval enhances the discrimination and comprehensiveness of evaluations. Employing LLMs as evaluators significantly reduces costs while maintaining a commendable level of reliability.

6.2 RQ3. Results for Baselines by DeepCRCEval

Utilizing DeepCRCEval, as delineated in Section 5.1, we reassessed the quality of comments generated by different CRCGs. The scoring and ranking outcomes from both evaluator types were averaged to derive final results.

Scoring The scoring results, presented in the first part of Table 6, demonstrate a notable superiority of our newly proposed baseline, LLM-Reviewer, across almost all evaluation aspects. This performance advantage is attributed to LLM-Reviewer’s direct alignment with the objectives of the code review task, guided by specific quality criteria. In contrast, previous methods, steered by indirect text similarity metrics, often underperformed in several aspects, failing to achieve holistic excellence in comment quality.

Table 6. Average Scoring results by DeepCRCEval (both human and LLM evaluators).

Method Name	Scoring (1-10, Higher = Better) (H. represents Humans and M. represents LLMs)																	
	C1		C2		C3		C4		C5		C6		C7		C8		C9	
	H.	M.	H.	M.	H.	M.	H.	M.	H.	M.	H.	M.	H.	M.	H.	M.	H.	M.
Tufano	8.33	6.36	4.97	3.65	1.40	3.44	1.87	3.35	2.03	3.54	1.87	3.27	2.13	3.57	2.13	4.26	8.87	8.16
CommentFinder	8.27	6.52	2.23	2.77	1.43	2.76	1.27	2.54	1.20	2.71	1.23	2.53	1.30	2.76	1.30	3.43	8.83	8.29
CodeReviewer	7.83	7.13	5.93	3.68	1.00	3.38	2.27	3.29	1.77	3.53	1.90	3.17	2.50	3.54	3.20	4.35	9.50	8.37
AUGER	8.33	4.98	1.93	2.27	1.00	2.16	1.00	2.04	1.13	2.09	1.17	2.04	1.00	2.19	1.07	2.84	9.17	8.43
CCT5	9.63	7.53	3.30	2.90	1.00	2.66	1.23	2.55	1.03	2.60	1.00	2.51	1.23	2.77	1.70	3.79	9.83	9.27
LLM-Reviewer	9.97	9.24	10.00	9.55	9.67	9.17	9.80	9.40	9.83	9.12	9.37	8.89	9.87	9.32	9.90	9.59	9.97	8.23

Table 7. Average ranking results by DeepCRCEval (both human and LLM evaluators).

	Tufano et al.	CommentFinder	CodeReviewer	AUGER	CCT5	LLM-Reviewer
Humans	2.77		4.77	2.67	5.77	4.03
LLMs	3.3		4.17	3.33	5.19	4.00
						1
						1

Ranking The ranking results, depicted in the second part of Table 7, unequivocally place LLM-Reviewer at the top, as acknowledged by both human and LLM evaluators. Following LLM-Reviewer, Tufano and CodeReviewer were closely matched in quality, occupying the second and third positions, respectively. CCT5 was ranked fourth, CommentFinder fifth, and AUGER lagged at the sixth position.

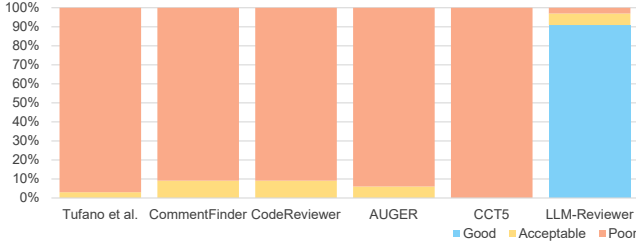


Fig. 5. User feedback ratings distribution for “Good”, “Acceptable”, and “Poor”.

Response to RQ3: LLM-Reviewer, guided by direct task objectives and explicit criteria, excels in generating high-quality review comments, surpassing previous models.

7 Discussion

7.1 Implications

Our findings highlight the limitations of existing text similarity metrics in evaluating code review comment generation. Our proposed framework, DeepCRCEval, demonstrates superior ability in discriminating between high and low-quality reviews, offering a more comprehensive assessment. Additionally, our new baseline, LLM-Reviewer, guided by direct targets and specific criteria, outperforms previous models.

To assess its practical utility, we tested a web application developed using the Gradio library with 5 industry developers. Users were instructed to input code snippets, and the application provided generated comments from each model. User feedback on total 66 cases, categorized as “Good” (I), “Acceptable” (II), or “Poor” (III), is summarized in Figure 5. The feedback corroborates our findings: comments generated by LLM-Reviewer, noted for their quality and effectiveness, received more “Good” ratings. This user input highlights the potential direction for future research, emphasizing the need to delve into deeper, domain-specific features rather than treating code review as a standard text-to-text task.

Implications: For code review comment generation, researchers should prioritize the fundamental objectives of code reviews instead of focusing merely on textual similarity. Incorporating the specific goals of code reviews directly into the training methodologies seems promising.

7.2 Case Study

For a qualitative comparison, we present two case studies in Table 8—one highlighting a defect and another focusing on code improvement. These cases con-

trast high-quality comments from LLM-Reviewer with those of lower quality from other baselines. Comments from baseline models often suffer from a lack of relevance or generality, potentially confusing developers. This deficiency may stem from their reliance on indirect text similarity metrics, leading to suboptimal outcomes. Conversely, LLM-Reviewer’s comments are notably superior, providing precise, clear explanations of problems, their causes, and suggested solutions, corroborating our previous findings.

Table 8. Case studies comparing baseline comments.

Case 1: DeadLock Issue	
Code: <pre> public class SyncExample { private final Object lock1 = new Object(); private final Object lock2 = new Object(); public void method1() { synchronized(lock1) { synchronized(lock2) { System.out.println("Lock1 then Lock2"); } } } public void method2() { synchronized(lock2) { synchronized(lock1) { System.out.println("Lock2 then Lock1"); } } } } </pre>	
Source	Comment
Tufano et al.	This is exactly the same class, you can't have two methods, one with the same name, and another with the same name.
CommentFinder	May want to get rid of this System.out running loose here in this test case.
CodeReviewer	Why is this needed?
AUGER	yes i think we should have this in the hoosier state lock block
CCT5	Remove this line.
LLM-Reviewer	In "SyncExample" class, "method1" and "method2" obtain locks in a different order. If they are called concurrently, it can lead to a deadlock where each method is waiting for the other to release a lock. Always acquire locks in the same order to prevent deadlocks.
Case 2: Map Performance Issue	
Code: <pre> public void processMap(Map<String, Integer> map, String key) { if (map.containsKey(key)) { Integer value = map.get(key); System.out.println(value); } } </pre>	
Source	Comment
Tufano et al.	Can we make this a logger?
CommentFinder	> I would rather use the first version and avoid the cast to M. Is that OK? Please cast (M) map.put(key, value). We save an additional Tuple instance which scales for many calls. I see our collections as low-level, they should be as performant as possible. A cast is fast.
CodeReviewer	Remove this file, its irrelevant to this PR.
AUGER	why do we need this
CCT5	Please remove this.
LLM-Reviewer	The "processMap" method performs two lookups: one for "containsKey" and another for "get". This is inefficient. Instead, use "get" directly and check if the result is null. It performs the operation in a single lookup, improving efficiency.

Case Study: LLM-Reviewer’s high-quality comments deliver concise, clear explanations of issues, their roots, and potential fixes, contrasting with the irrelevant or generic comments from other models.

7.3 Threats to Validity

Several factors are crucial for assessing the validity of our findings. Firstly, our selection of large language models (LLMs), specifically GPT-4, was a deliberate decision. For DeepCRCEval, GPT-4 was chosen to emulate human evaluators because of its advanced capabilities. Likewise, GPT-4 served as the foundation for LLM-Reviewer due to its status as a leading and representative LLM. Another consideration is the focus on the Java programming language for our code review task. While Java has specific characteristics that differ from other languages, it is the most commonly used language in prior research, making it suitable for comparison. Finally, the scope of our reevaluation needs acknowledgment. We used graduate computer science students as proxies for actual developers in human evaluations. These students have significant programming experience, making them a reasonable approximation. Additionally, due to the high costs of manual analysis, our study covered a relatively small sample size. To mitigate this, we used LLM evaluators for broader analysis and compared the concordance between human and LLM evaluations, ensuring a thorough and robust assessment.

However, using LLMs to evaluate LLMs still potentially introduces bias, which is why we also used human evaluators. The introduction of LLM-Reviewer aims to highlight the significant room for improvement in existing CRCGs. The results show that they score particularly low compared to LLM-Reviewer, a discrepancy even more pronounced when by humans, which cannot be wholly attributed to bias.

8 Related Work

The automation and evaluation of machine code review comments are recent developments, aligning with longstanding research interests in assessing the quality of human-written comments. Traditional evaluations predominantly focused on the “usefulness” of comments. Early methods, exemplified by Bosu et al. [3], employed decision trees and hand-crafted rules to categorize comments as “useful” or “not useful”, often based on subsequent code modifications or “wontfix” labels. Rahman et al. [23] refined this approach by emphasizing comments’ “change-triggering” characteristics, incorporating textual content and reviewer experience into their predictive models. Hasan et al. [9] expanded this further by integrating additional features from review contexts and reviewer backgrounds. A notable advancement came with Yang et al. [33], who introduced a BERT-based scoring system across four dimensions (emotion, question, evaluation, and suggestion), marking a shift towards a more detailed and explanatory evaluation, beyond extensive feature engineering.

However, Yang et al.’s methodology diverges from ours in two key respects. First, their model assesses human-generated comments, whereas our focus is on **machine**-generated comments aimed at enhancing code review quality. This necessitates a more granular evaluation, emphasizing clarity and effectiveness in addressing actual defects or improvements, as detailed in Section 2.1. Second, our approach demands a deeper evaluation, analyzing the interaction between **code**

and comment pairs, while previous research primarily targeted comments alone. Without including code as a target, it is impossible to judge perspectives related to actual issues in the code. Finally, we utilize the emergent abilities of **LLMs** like in-context learning. Earlier models like BERT lacked the depth required for such analysis, but recent LLM advancements, especially GPT-4, have shown near-human comprehension in understanding both code and language. Hence, DeepCRCEval incorporates LLM evaluators to complement human evaluation, balancing reliability with reduced time and cost.

9 Conclusion

This study challenges the prevailing evaluation methodology for CRCGs, arguing that text similarity metrics like BLEU and ROUGE-L are inadequate due to the questionable quality and validity of benchmark comments. As a solution, we introduced DeepCRCEval, a framework directly addressing developers' concerns, and LLM-Reviewer, a lightweight, training-free baseline for CRCG evaluation. LLM-Reviewer, guided by clear and direct task goals and criteria, contrasts with methods relying on text similarity for training.

Our empirical findings suggest that CRCGs might overstate their improvements when focused on text similarity metrics, often producing comments that are irrelevant or overly generic. In contrast, LLM-Reviewer demonstrates the ability to provide clear, concise explanations of issues, their causes, and potential solutions, even without specific training. DeepCRCEval offers superior discrimination and comprehensiveness compared to previous metrics, and significantly reduces costs while maintaining reliable evaluation standards when employing LLMs as alternative evaluators. Our work lays a foundation for task-specific evaluations for code review comment generation, and highlights that future researchers should not neglect the original objectives of the code review.

Acknowledgments. This work was supported by the National Key Research and Development Program of China (No. 2023YFB3307202) and the Alliance of International Science Organizations Collaborative Research Program (No. ANSO-CR-KP-2022-03).

References

1. Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE). pp. 712–721 (2013). <https://doi.org/10.1109/ICSE.2013.6606617>
2. Bosu, A., Carver, J.C., Bird, C., Orbeck, J., Chockley, C.: Process aspects and social dynamics of contemporary code review: Insights from open source development and industrial practice at microsoft. *IEEE Transactions on Software Engineering* **43**(1), 56–75 (2017). <https://doi.org/10.1109/TSE.2016.2576451>
3. Bosu, A., Greiler, M., Bird, C.: Characteristics of useful code reviews: An empirical study at microsoft. In: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories. pp. 146–156 (2015). <https://doi.org/10.1109/MSR.2015.21>

4. Creswell, J.W., Creswell, J.D.: Research design: Qualitative, quantitative, and mixed methods approaches. Sage publications (2017)
5. Dalkey, N., Helmer, O.: An experimental application of the delphi method to the use of experts. *Management science* **9**(3), 458–467 (1963)
6. Dubois, Y., Li, X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P., Hashimoto, T.B.: Alpacaform: A simulation framework for methods that learn from human feedback. arXiv preprint arXiv:2305.14387 (2023)
7. Fagan, M.: Design and code inspections to reduce errors in program development. In: *Software pioneers*, pp. 575–607. Springer (2002)
8. Gupta, A., Sundaresan, N.: Intelligent code reviews using deep learning. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD’18) Deep Learning Day* (2018)
9. Hasan, M., Iqbal, A., Islam, M.R.U., Rahman, A.I., Bosu, A.: Using a balanced scorecard to identify opportunities to improve code review effectiveness: An industrial experience report. *Empirical Softw. Engg.* **26**(6) (nov 2021). <https://doi.org/10.1007/s10664-021-10038-w>, <https://doi.org/10.1007/s10664-021-10038-w>
10. Hong, Y., Tantithamthavorn, C., Thongtanunam, P., Aleti, A.: Commentfinder: a simpler, faster, more accurate code review comments recommendation. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 507–519 (2022)
11. Hua, Z.H., Yang, L., Lu, J.Y., Zuo, C.: Survey on code review automation research. *Ruan Jian Xue Bao/Journal of Software* **35**(7), 3265–3290 (2024) (in Chinese). <http://www.jos.org.cn/1000-9825/7112.htm>
12. Kononenko, O., Baysal, O., Godfrey, M.W.: Code review quality: How developers see it. In: *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. pp. 1028–1038 (2016). <https://doi.org/10.1145/2884781.2884840>
13. Li, D., Jiang, B., Huang, L., Beigi, A., Zhao, C., Tan, Z., Bhattacharjee, A., Jiang, Y., Chen, C., Wu, T., et al.: From generation to judgment: Opportunities and challenges of llm-as-a-judge. arXiv preprint arXiv:2411.16594 (2024)
14. Li, L., Yang, L., Jiang, H., Yan, J., Luo, T., Hua, Z., Liang, G., Zuo, C.: Auger: automatically generating review comments with pre-training models. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 1009–1021 (2022)
15. Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P., Hashimoto, T.B.: AlpacaEval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval (2023)
16. Li, Z., Lu, S., Guo, D., Duan, N., Jannu, S., Jenks, G., Majumder, D., Green, J., Svyatkovskiy, A., Fu, S., et al.: Automating code review activities by large-scale pre-training. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 1035–1047 (2022)
17. Lin, B., Wang, S., Liu, Z., Liu, Y., Xia, X., Mao, X.: Cct5: A code-change-oriented pre-trained model. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (2023)
18. Lin, C.Y.: Rouge: A package for automatic evaluation of summaries. In: *Text summarization branches out*. pp. 74–81 (2004)
19. Lindlof, T.R., Taylor, B.C.: *Qualitative communication research methods*. Sage publications (2017)
20. Lu, J., Li, Z., Shen, C., Yang, L., Zuo, C.: Exploring the impact of code review factors on the code review comment generation. *Automated Software Engineering* **31**(2), 71 (2024)

21. Lu, J., Yu, L., Li, X., Yang, L., Zuo, C.: Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). pp. 647–658. IEEE (2023)
22. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th annual meeting of the Association for Computational Linguistics. pp. 311–318 (2002)
23. Rahman, M.M., Roy, C.K., Kula, R.G.: Predicting usefulness of code review comments using textual features and developer experience. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR). pp. 215–226 (2017). <https://doi.org/10.1109/MSR.2017.17>
24. Ray: Aviary: Study stochastic parrots in the wild. <https://github.com/ray-project/aviary> (2023)
25. Rigby, P.C., Bird, C.: Convergent contemporary software peer review practices. In: Proceedings of the 2013 9th joint meeting on foundations of software engineering. pp. 202–212 (2013)
26. Rigby, P.C., German, D.M., Cowen, L., Storey, M.A.: Peer review on open-source software projects: Parameters, statistical models, and theory. *ACM Transactions on Software Engineering and Methodology (TOSEM)* **23**(4), 1–33 (2014)
27. Rigby, P.C., German, D.M., Storey, M.A.: Open source software peer review practices: a case study of the apache server. In: Proceedings of the 30th international conference on Software engineering. pp. 541–550 (2008)
28. Sadowski, C., Söderberg, E., Church, L., Sipko, M., Bacchelli, A.: Modern code review: a case study at google. In: Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice. pp. 181–190 (2018)
29. Shan, Q., Sukhdeo, D., Huang, Q., Rogers, S., Chen, L., Paradis, E., Rigby, P.C., Nagappan, N.: Using nudges to accelerate code reviews at scale. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 472–482 (2022)
30. Siow, J.K., Gao, C., Fan, L., Chen, S., Liu, Y.: Core: Automating review recommendation for code changes. In: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER). pp. 284–295. IEEE (2020)
31. Tufano, R., Masiero, S., Mastropaolo, A., Pascarella, L., Poshyvanyk, D., Bavota, G.: Using pre-trained models to boost code review automation. In: Proceedings of the 44th International Conference on Software Engineering. pp. 2291–2302 (2022)
32. Tufano, R., Pascarella, L., Tufano, M., Poshyvanyk, D., Bavota, G.: Towards automating code review activities. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). pp. 163–174. IEEE (2021)
33. Yang, L., Xu, J., Zhang, Y., Zhang, H., Bacchelli, A.: Evacrc: Evaluating code review comments. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. p. 275–287. ESEC/FSE 2023, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3611643.3616245>, <https://doi.org/10.1145/3611643.3616245>
34. Yang, X., Kula, R.G., Yoshida, N., Iida, H.: Mining the modern code review repositories: A dataset of people, process and product. In: Proceedings of the 13th International Conference on Mining Software Repositories. pp. 460–463 (2016)
35. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al.: Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* (2023)

Open Access. This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution, and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

